Castillo Vizcarra Ihrahn Grupo 6

Corrección del Primer Parcial MPOO

Pregunta 0.1

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código. ¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

Abstracción es quedarse solo con las características esenciales de un objeto para el problema que se quiere resolver, y dejar fuera lo que no importa. El ejemplo dado fue que para calcular el área de un círculo conservo el radio e ignoro el color, ya que no es necesario para sacar su área, si usara un programa de dibujo sí sería útil saber el color.

Pregunta 0.2

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

Sirve para inicializar los atributos de la instancia en el momento de crearla con new. En su declaración se distingue porque lleva el mismo nombre que la clase y no tiene tipo de retorno.

Pregunta 0.3

¿Qué impide final de una variable? Si guarda referencia a arreglo, ¿impide cambiar sus elementos?

No impide cambiar los elementos del arreglo, porque el arreglo no es final, solo la variable que lo apunta. "final" impide reasignar la variable después de inicializarla, no se puede cambiar la referencia para que apunte a otro objeto.

Pregunta 0.4

¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

Porque == compara identidad: revisa si las dos referencias apuntan al mismo objeto en memoria, no si el texto es igual. Dos String con el mismo texto pueden ser objetos distintos y == da falso. Para comparar el texto usaría a.equals(b).

PRINTAFORM

Pregunta 0.5

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

while evalúa la condición antes de ejecutar el cuerpo, así que si es falsa desde un inicio da cero vueltas. do-while ejecuta el cuerpo y después evalúa la condición, así que da al menos una vuelta aunque la condición sea falsa desde el inicio. Ninguno de los dos evita bucles infinitos pues eso depende de la condición.

R1. ¿Qué imprime?

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int [] [] tabla = {{9, 4, 7}, {2, 6, 8}};
int [] fila = tabla [0];
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
    total += tabla [0] [i];
    System.out.println (i + ":" + total);
}
System.out.println ((double) (total / 2));
System.out.println ("P" + fila[0] + tabla[1][0]);
```

Salida
0:4
1:6
2:9
4.0
P42

Sale así porque int [] fila = tabla[0]; no copia el arreglo, solo copia la referencia. fila y tabla [0] son el mismo arreglo, así que lo que cambia fila[i] también lo ve tabla[0] [i]. Además, fila [i] / 2 es división entre enteros. Inicio: total = 0, fila = tabla [0] = {9, 4, 7}.

Vuelta	i	fila[i] antes	fila[i] /2	total	println
1	0	9	4	0+4=4	0:4
2	1	4	2	4+2=6	1:6
3	2	7	3	6+3=9	2:9

PRINTAFORM

Cuando i=3 ya no se cumple 3 < 3, termina el ciclo. Ahora tabla[0] = {4, 2, 3} y total = 9

println	Calculo	Sale
4°	total / 2 = 9/2 = 4	4.0
5°	"P" + tabla[1] = "P4" "P4" + tabla[1][0] = "P42"	P42

R2 a) · ¿Compila, truena o funciona?
Incisos independientes: a) b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).
int [] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
    total += datos[i];
System.out.println(total);
Compila pero truena. Porque datos tiene 2 elementos, así que sus índices válidos son 0 y 1. La condición i <= datos.length deja que i valga 2, y en esa vuelta la línea total += datos[i]; accede a datos[2], que no existe. Java lanza ArrayIndexOutOfBoundsException al ejecutar (el compilador no revisa los índices, por eso sí compila). Luego nace de esto i=0 → total=3; i=1 → total=9; i=2 → datos[2] truena y el println nunca se ejecuta.
La solución sería usar i < datos.length.

R2 b) - ¿Compila, truena o funciona?
Incisos independientes: a) b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).
Integer piezas = 6;
System.out.println(piezas + 2);
System.out.println("P" + piezas);

PRINTAFORM

Funciona y la salida es 8 / P6, esto porque Integer es una clase envoltorio de int. En primeras + 2 no hay texto, así que Java desempaqueta el Integer a int (unboxing), suma 6 + 2 e imprime 8. En "P" + primer uno de los operandos es un String, así que + concatena: el Integer se convierte a texto "6" y queda "P6".

R2 c) · ¿Compila, truena o funciona?
Intenta independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y de la causa en una frase. Si funciona, escribe la salida (separa líneas con /).
static int medir (int valor) {
    if (valor > 0) return valor;
}
No compila porque el método declara que devuelve int, así que todos los caminos deben terminar en un return. Si valor es 0 o negativo, if es falso, el método llega al } final sin devolver nada y el compilador marca missing return statement. Java no tiene endif; el compilador revisa todos los caminos posibles, aunque no se ejecuten.

R3 · Modelado: base, reto y justificación
Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender(), y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.
enum Estado { APAGADO, ENCENDIDO, PARPADEO }
Base. Declara al menos dos atributos de instancia, uno static y una constante final (¿también static?). Completa el constructor y dos métodos: cambia y consulta.
Reto. ¿Qué tipo tiene el atributo de estado y con qué valor nace?
Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.
Cuerpos de la 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

PRINTAFORM

```
class Lampara {
    private final String etiqueta                    // instancia, final
    private Estado estado;                           // instancia/mutable
    private static int totalCreadas = 0;             // static, mutable
    private static final Estado ESTADO_INICIAL = Estado.APAGADO; // constante static final

    Lampara (String etiqueta) {
        this.etiqueta = etiqueta;
        this.estado = ESTADO_INICIAL;
        totalCreadas++;
    }

    void encender() {
        estado = Estado.ENCENDIDO;
    }

    Estado consultaEstado() {
        return estado;
    }
}
```

Reto: El atributo de estado es de tipo Estado (el enum, no String ni int) y nace con el valor Estado.APAGADO, que se toma de la constante ESTADO_INICIAL.

Justificaciones:
- Tipos: etiqueta a String porque es texto; estado a Estado (enum) porque solo admite los 3 valores válidos; y totalCreadas es int porque es un conteo.
- Instancia/clase: etiqueta y estado son de instancia, totalCreadas es static.
- final o mutable: etiqueta a final, estado es mutable, y totalCreadas es mutable.
- Constantes: ESTADO_INICIAL es static final.
- Constructor: recibe la etiqueta, deja el estado en apagado y suma 1 al contador.
- Métodos: encender() es void y cambia el estado de esa lámpara (this); consultaEstado() devuelve el Estado actual sin modificar nada.

R4. Código con repetición.

Devuelve la posición de la menor temperatura usada hoy; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

static int posicionMinima(int[] temperaturas)

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1. {7} → 0. {-2, -8, -1} → 1

Escribe el método completo. No imprimas ni leas del teclado.

```
static int posicionMinima(int[] temperaturas) {
    int posicion = 0;
    for (int i = 1; i < temperaturas.length; i++) {
        if (temperaturas[i] < temperaturas[posicion]) {
            posicion = i;
        }
    }
    return posicion;
}
```

Posición guarda la posición de la menor temperatura vista hasta ahora. Empieza en 0, y el ciclo empieza en i = 1 porque no hay que comparar lo consigo mismo. Uso < estricto para que, si hay un valor repetido, se quede la primera posición.

Ejemplo	Seguimiento	Devuelve
{8, 3, 5}	posicion = 0 (vale 8). i = 1: 3 < 8 sí → posicion = 1. i = 2: 5 < 3 no → sigue 1.	1
{7}	posicion = 0. El ciclo empieza en i = 1 pero 1 < 1 es falso, no da ninguna vuelta.	0
{-2, -8, -1}	posicion = 0 (vale 2). i = 1: -8 < -2 sí → posicion = 1. i = 2: -1 < -8 no → sigue 1.	1

Con negativos funciona porque comparo contra el valor del arreglo (temperaturas[posicion]) y no contra un 0 inicial inventado.

PRINTAFORM

RS · Codigo con selección

Un equipo decide su mantenimiento según los ciclos de uso. Un valor menor o igual que 0 es "INVALIDO". Para los positivos, aplica la primera regla que corresponda en el orden de la tabla. Usa if / else-if / else. No imprimas ni leas del teclado.

static String mantenimiento(int ciclos)

Regla · valor recibido	Texto devuelto
Menor o igual que 0	"INVALIDO"
Divisible entre 24	"COMPLETO"
Si no, divisible entre 12	"REVISION"
Si no, divisible entre 4	"BASICO"
Cualquier otro	"NINGUNO"

```
static String mantenimiento(int ciclos) {
    if (ciclos <= 0) {
        return "INVALIDO";
    } else if (ciclos % 24 == 0) {
        return "COMPLETO";
    } else if (ciclos % 12 == 0) {
        return "REVISION";
    } else if (ciclos % 4 == 0) {
        return "BASICO";
    } else {
        return "NINGUNO";
    }
}
```

El orden importa porque Java revisa de arriba hacia abajo y entra en la primera condición verdadera. Primero va <= 0, luego 24 antes que 12 y que 4. El else final es el caso de ninguna regla, todos los caminos terminan en return.

PRINTAFORM

Renglón	valor ejemplo	Secuencia	Devuelve
Menor o igual que 0	0 o -5	0 <= 0 es verdadero y entra al if	"INVALIDO"
Divisible entre 24	48	48 <= 0 no; 48 % 24 = 0 sí	"COMPLETO"
Divisible entre 12	36	36 <= 0 no; 36 % 24 = 12 no; 36 % 12 = 0 sí	"REVISION"
Divisible entre 4	20	20 <= 0 no; 20 % 24 = 20 no; 20 % 12 = 8 no; 20 % 4 = 0 sí	"BASICO"
Cualquier otro	7	7 <= 0 no; 7 % 24 = 7, 7 % 12 = 7, 7 % 4 = 3, ninguno es 0 → else	"NINGUNO"